

Session 0 - Overview

Overview

We will build a recommendation system with necessary backends (feature store, model monitoring etc.) for e-commerce recommendation. We will build a two-stage ranking solution to generate item recommendations for customers. It will support incremental training, advanced features and model arch.

recommendation system features:

- sparse feature (categorical feature) -> hashing -> embedding
- dense feature (counting feature)-> feature vector
- sequential feature -> advanced feature

Dataset

Let's start with the Amazon review dataset (<https://amazon-reviews-2023.github.io/>). The dataset contains two parts:

- user centric sets - product reviews from a specific user
- item centric sets - aggregated user reviews for a specific product

We will build models to generate recommendations at user level.

Overall Components

We will build the following components (with revision and iteration in later sessions):

- item-indexed storage (python dictionary -> sql database)
- retrieval (rule-based -> graph-based/model-based)
- ranking (MLP -> Transformer-like, generative)
- calibration & evaluation
- other system components (incremental training, feature store, feature pipeline)

Requirements

- Start with simple workable solution; let's establish a baseline first
- You can use python packages with common neural networks (sklearn, huggingface transformer etc.), but extra credits if you can build things with basic pytorch from scratch

Tasks

- Get familiar with the dataset, Pytorch framework, sql syntax and sqlite3
- Get familiar with common evaluation metrics for recommendation systems: AUC, **gAUC**, MRR, nDCG@m, precision, recall
- evaluator(prediction, labels) -> metric value

Matrix multiplication:

- vectorized
 - dense matrix multiplication (label = [1, 0, 0, padding,], model_pred = [0.3, 0.5, padding])
 - for each sample,
 - compute accuracy(sample, gt) etc
 - avg
 - eval(label, model_pred) -> [eval_metrics, padding]
 - jagged tensor [1, 0, 0, 1, 1, 1, 1], len = [3, 4] -> jagged tensor computation to compute matrix, feedforward

Next session: dataset analysis and data preprocessing

Session 1 - Data preprocessing & stats analysis

Dataset stats analysis

- Play around with the datasets. Pick a few categories with reasonable volume (Book, electronic etc.), collect a few metrics
 - P20, P50, P90, P99 on number of reviews per item, per user
 - preproc: filter on a threshold: product must have more than a THRESHOLD reviews to be included
 - user: THRESHOLD on min reviews avail
 - Bought together hit rate: what % of users have reviews of products recommended in bought_together?
 - Cross-category browsing & purchase:
 - Distribution of ratings and engagements
- Preliminary modeling
 - Can you build a model to correlate review content to rating? (this along can be a full-scale project, but since we are focusing on recsys for this project, we will not dive super deep into this part)
 - Compare model choices:
 - MLP
 - Regression model - word count, key word; [text_feature] -> regression model -> score [1,5]
 - Multiclass classification (5 class- 1, 2, 3, 4, 5)
 - compare accuracy among models, and among categories
 - text encoding based prediction

Data preproc

You can take reference in “Common Data Processing” section:

https://amazon-reviews-2023.github.io/data_processing/0core.html

Dataset cleanup

- Handling missing values
 - investigate the null rate for item and user features
 - extrapolate values based on feature states across dataset or fill a default values
- Deduplicate records
 - same product name, different product id
 - keep the product corresponds to parent product id, or the one with more reviews
- Consolidating items of product-parent product into a single item, represented by the parent asin.
- Label preparation
 - convert rating number into binary labels (≥ 3 as positive, else negative)

- analyze label distribution for items and users

Feature store construction

Parse the json and build feature stores (can be dataframe or python dictionary for now) for user and item:

- Key will be user_id for user feature store, and product_id for item feature store
- collect features (numerical, categorical, text)

Preparing train/test split and samples

- Split user (with reviews on the category) into 0.7, 0.1, 0.2 for train, val, test sets
- For each user, split historical interactions based on timestamp
 - For N item interaction, split N-2 items for training sequence, last 2 for validation and test
- Verify label distribution and feature distribution are consistent between train, val and test set.

Next session: Retrieval + baseline

Session 2 - Retrieval + baseline

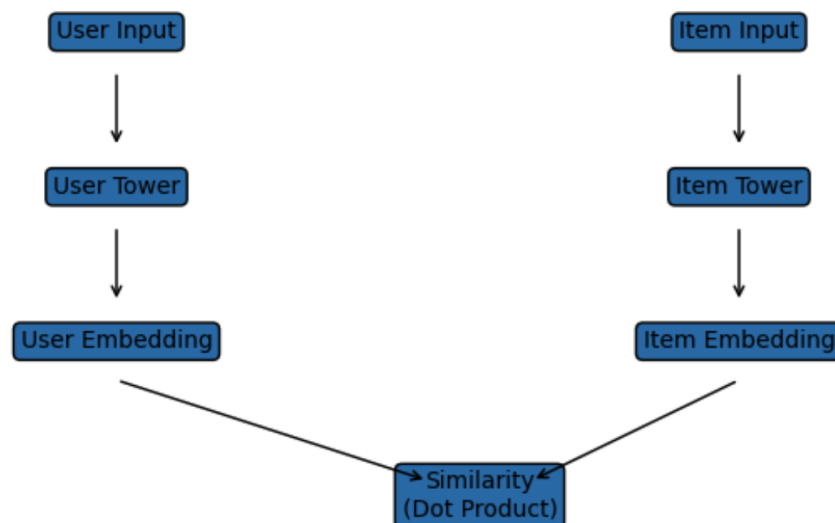
Retrieval

The retrieval model serves as a fast candidate generation system for downstream ranking models to rank. Usually, retrieval generates candidates at the scale of hundreds to thousands of items.

Implementation:

Build 3 versions of retrievals (from easy to complex) - generate 1k candidates per user:

1. popularity based retrieval
2. rule-based matching: retrieve items based on user metadata. i.e. categories a user engage with the most; popular items in user age bucket/country etc.
3. Two-tower model to co-train user and item embeddings
 - a. user feature: some metadata (country, age, gender etc.) hashed to id, then emb look up
 - b. item input: item related metadata (store, seller etc.) hashed to id, then look up



Label preparation

Refer to session 1 for positive / negative samples

Evaluation metrics:

- Recall@K (K=50, 100)
- Precision@K
- cross entropy loss,

Baseline

For the baseline model, let's build a plain deeper MLP model, using retrieval score + user features + item features + retrieval source.

- input: <user_id, item_id> pairoutput: prediction score [0, 1] represents likelihood of user will engage with the item

Report:

- Cross entropy loss of the train / val / test metrics, for different combination of retrieval model
- Recall, precision, AUC metrics

Next session: Evaluation & calibration framework

Session 3 - baseline ranking model

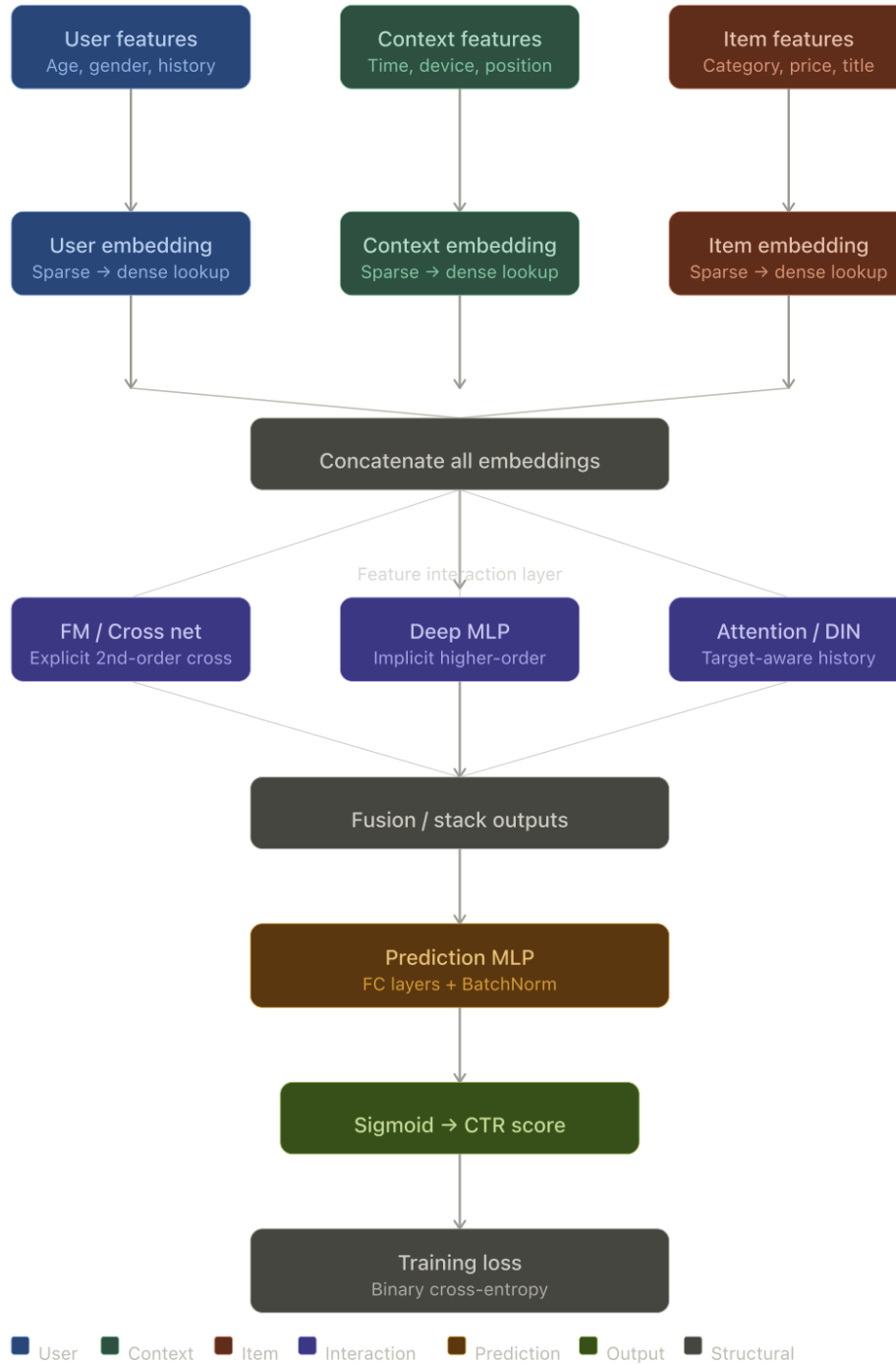
Ranking Model

The ranking model serves as the central model handling the recommendation prediction. In most scenarios, the model is the heaviest model in the entire recommendation pipelines.

Baseline model arch

4/19/26, 9:44 PM

CTR Ranking Model Architecture Flowchart



Unifying candidates from different sources

- Dedup but keep <source, score> pairs
- Leave proportion from different retrieval source as a config json

Feature List

- User side
 - Metadata from user profile
 - User side counting features
- Item side
 - metadata from item
 - item side counting features
- Interaction
 - retrieval scores
 - retrieval sources
 - user x item category counts in past 1/7/15 days
 - matching feature (0/1) on user x item interaction in the past

Evaluation

- Train/val/test loss
- Feature importance analysis
- Ranking performance
 - precision, recall @ k
- Auroc, auprc

ML goal

ML learning goal:

- organic content (user create items)-> finish view, engagement metrics (comment, like, repost)
- Ads -> CTR (click through rate), CTR over impression ()
- friend recommendation -> sent rate, accept rate